

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО»
(Университет ИТМО)

КУРСОВАЯ РАБОТА

“Разработка Android-приложения "Мессенджер" с новостной лентой”

Выполнила:

Гуторова Инна Владимировна, К3441

Преподаватель:

Шатинский Григорий Сергеевич

Санкт-Петербург

2026

ВВЕДЕНИЕ.....	3
1 Анализ требований и постановка задачи.....	5
1.1 Функциональные требования.....	5
1.2 Нефункциональные требования.....	6
2 Архитектура приложения и навигация.....	7
3 Управление состоянием приложения и архитектура MVVM.....	9
4 Работа с данными, сеть и офлайн-доступ.....	12
5 Экран ленты сообщений и пользовательские сценарии.....	14
6 Улучшение интерфейса и фоновая синхронизация.....	17
ЗАКЛЮЧЕНИЕ.....	19

ВВЕДЕНИЕ

В условиях активного развития мобильных технологий программные приложения для платформы Android играют ключевую роль в повседневной деятельности пользователей. Современные Android-приложения предъявляют высокие требования не только к внешнему виду и удобству интерфейса, но и к внутренней архитектуре, устойчивости к ошибкам, корректной работе с жизненным циклом компонентов и обработке фоновых процессов. В связи с этим особую значимость приобретает изучение архитектурных подходов и инструментов, рекомендуемых платформой Android.

Целью данной курсовой работы является разработка Android-приложения типа «Мессенджер», демонстрирующего поэтапное формирование архитектуры, управление состоянием пользовательских данных, работу с сетевыми источниками и локальным хранилищем, а также реализацию фоновой синхронизации. Приложение создавалось последовательно в рамках нескольких этапов разработки, каждый из которых расширял функциональные возможности и архитектурную сложность проекта.

В процессе выполнения курсовой работы были поставлены следующие задачи: разработать пользовательский интерфейс с использованием Activity и Fragment; реализовать навигацию между экранами с применением Android Navigation Component; внедрить архитектуру MVVM для управления состоянием экранов; обеспечить сохранение данных при изменении конфигурации устройства; организовать загрузку данных из сети и их хранение в локальной базе данных; реализовать устойчивую работу приложения при отсутствии интернет-соединения; настроить фоновую синхронизацию данных и

систему уведомлений; обеспечить корректную обработку ошибок и разрешений операционной системы.

Итоговое приложение представляет собой многоэкранный Android-клиент с навигацией через нижнюю панель, поддержкой светлой и тёмной темы, офлайн-доступом к данным и периодическим обновлением информации в фоновом режиме. Реализация выполнена с соблюдением принципов разделения ответственности и расширяемости, что делает архитектуру пригодной для дальнейшего развития.

1 Анализ требований и постановка задачи

1.1 Функциональные требования

Функциональные требования к приложению формировались поэтапно и в итоговой версии включают в себя требования всех этапов разработки.

Разрабатываемое приложение должно:

- содержать одну главную активность, выступающую в качестве точки входа в приложение;
- обеспечивать навигацию между экранами с использованием Bottom Navigation;
- включать следующие основные экраны:
 - экран новостной ленты с отображением списка сообщений;
 - экран профиля пользователя с возможностью редактирования персональных данных;
 - экран настроек приложения;
- использовать стандартные средства Android Navigation (NavController и Navigation Graph);
- реализовывать архитектуру MVVM для разделения ответственности между слоями приложения;
- хранить и обновлять состояние пользовательских данных с использованием ViewModel;
- обеспечивать реактивное обновление пользовательского интерфейса через LiveData;
- сохранять данные при изменении конфигурации устройства (например, повороте экрана);
- поддерживать загрузку данных из сетевого источника;

- обеспечивать локальное хранение данных;
- корректно обрабатывать ошибки сетевого взаимодействия;
- отображать пользователю уведомления об ошибках;
- отслеживать жизненный цикл Activity, Fragment и ViewModel с

выводом информации в лог.

Таким образом, приложение должно демонстрировать полный цикл работы типового Android-приложения - от построения интерфейса до обработки ошибок и управления состоянием.

1.2 Нефункциональные требования

К нефункциональным требованиям относятся:

- структурированность и читаемость исходного кода;
- возможность дальнейшего расширения функционала без изменения существующей архитектуры;
- устойчивость приложения к ошибкам и отсутствию интернет-соединения;
- соответствие рекомендациям Android по работе с жизненным циклом;
- понятный и интуитивный пользовательский интерфейс.

2 Архитектура приложения и навигация

Разработка приложения началась с формирования базовой структуры и общей архитектурной схемы. В качестве точки входа используется одна главная активность, что соответствует современным рекомендациям Android и позволяет централизовать управление навигацией и жизненным циклом экранов. Использование одной активности снижает сложность приложения и упрощает взаимодействие между его компонентами.

Внутри главной активности размещён контейнер `NavHostFragment`, отвечающий за отображение фрагментов. Навигация между экранами реализована с помощью `Android Navigation Component`, включающего `NavController` и граф навигации. Такой подход позволяет декларативно описывать переходы между экранами, упрощает поддержку проекта и делает структуру навигации наглядной.

```
override fun onCreate(savedInstanceState: Bundle?) {  
    super.onCreate(savedInstanceState)  
    setContentView(R.layout.activity_main)  
  
    val navController = findNavController( viewId = R.id.nav_host_fragment)  
    val bottomNav = findViewById<BottomNavigationView>( id = R.id.bottomNavigationView)  
    bottomNav.setupWithNavController(navController)
```

Рисунок 1 - Навигация в MainActivity

В приложении реализованы три основных экрана: лента сообщений, профиль пользователя и настройки. Каждый экран представлен отдельным `Fragment` и подключён к нижней панели навигации (`Bottom Navigation`). При выборе пункта меню происходит автоматическое переключение текущего фрагмента без необходимости ручного управления транзакциями.

Для повышения прозрачности работы приложения в процессе разработки было добавлено логирование жизненного цикла Activity и Fragment. В лог выводятся сообщения при создании, запуске, приостановке и уничтожении компонентов, что позволяет на практике изучить поведение приложения при смене экранов, сворачивании и пересоздании.

```
override fun onResume() {  
    super.onResume();  
    Log.i(tag = "Lifecycle", msg = "MainActivity onResume")  
}  
  
override fun onPause() {  
    super.onPause();  
    Log.i(tag = "Lifecycle", msg = "MainActivity onPause")  
}  
  
override fun onStop() {  
    super.onStop();  
    Log.i(tag = "Lifecycle", msg = "MainActivity onStop")  
}  
  
override fun onDestroy() {  
    super.onDestroy();  
    Log.i(tag = "Lifecycle", msg = "MainActivity onDestroy")  
}
```

Рисунок 2 - Логирование в MainActivity

3 Управление состоянием приложения и архитектура MVVM

После формирования навигационного каркаса приложения следующим этапом стала реализация архитектуры MVVM (Model–View–ViewModel). Данный архитектурный паттерн обеспечивает чёткое разделение ответственности между пользовательским интерфейсом и логикой работы с данными, что особенно важно для Android-приложений с динамическим жизненным циклом.

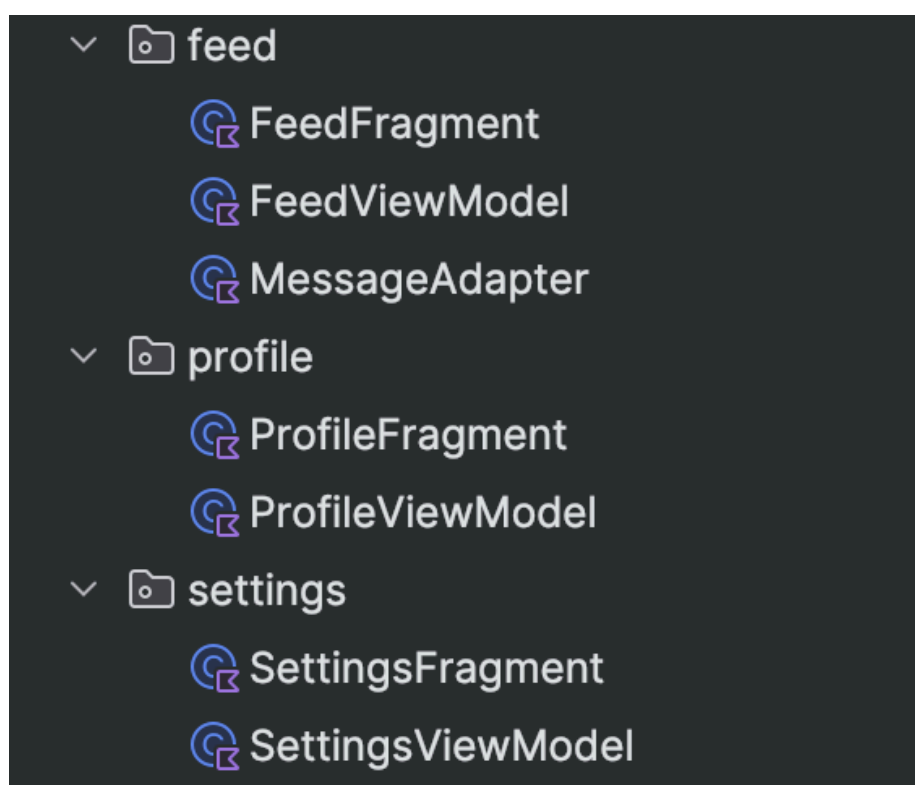


Рисунок 3 - Структура файлов с ViewModel

Слой View представлен Activity и Fragment, которые отвечают исключительно за отображение данных и обработку пользовательских действий. View не содержит бизнес-логики и не взаимодействует напрямую с источниками данных. Вместо этого View подписывается на состояние, предоставляемое ViewModel.

ViewModel используется для хранения состояния экранов и переживает изменения конфигурации устройства, такие как поворот экрана. В рамках приложения были реализованы отдельные ViewModel для экрана профиля, экрана настроек и экрана ленты сообщений. Это позволило изолировать логику каждого экрана и обеспечить сохранность данных при пересоздании интерфейса.

Для реактивного обновления пользовательского интерфейса применяется LiveData. Изменения состояния во ViewModel автоматически приводят к обновлению UI без необходимости вручную синхронизировать данные. Такой подход снижает количество ошибок и упрощает код фрагментов.

```
class SettingsViewModel : ViewModel() {  
    2 Usages  
    private val _isDarkMode = MutableLiveData(value = false)  
    2 Usages  
    val isDarkMode: LiveData<Boolean> get() = _isDarkMode  
  
    1 Usage  
    fun setDarkMode(enabled: Boolean) {  
        _isDarkMode.value = enabled  
    }  
}
```

Рисунок 4 - LiveData в SettingsViewModel

```
viewModel.isDarkMode.observe(viewLifecycleOwner) { enabled ->  
    themeSwitch.isChecked = enabled  
}  
  
themeSwitch.setOnCheckedChangeListener { _, isChecked ->  
    viewModel.setDarkMode(isChecked)  
    val mode = if (isChecked) AppCompatActivity.MODE_NIGHT_YES  
    else AppCompatActivity.MODE_NIGHT_NO  
    AppCompatActivity.setDefaultNightMode(mode)  
}
```

Рисунок 5 - Связь с ViewModel в SettingsFragment

В ViewModel также реализовано логирование жизненного цикла, включая момент создания и очистки ViewModel. Это позволяет наглядно увидеть, что ViewModel продолжает существовать при пересоздании Fragment и уничтожается только при завершении соответствующего жизненного цикла.

4 Работа с данными, сеть и офлайн-доступ

Одной из ключевых частей курсовой работы стала реализация полноценной работы с данными. Для получения сообщений используется сетевой API, доступ к которому осуществляется с помощью библиотеки Retrofit. Retrofit позволяет описывать HTTP-запросы в виде интерфейсов и автоматически преобразовывать ответы сервера в объекты Kotlin.

Все сетевые операции выполняются асинхронно с использованием корутин. Это гарантирует, что запросы не блокируют главный поток и пользовательский интерфейс остаётся отзывчивым даже при медленном соединении. В случае успешного запроса данные преобразуются в модели приложения и передаются на уровень хранения.

Для обеспечения офлайн-доступа в приложении используется база данных Room. Room представляет собой надстройку над SQLite и включает три основных компонента: Entity, DAO и Database. Entity описывает структуру таблицы сообщений, DAO содержит методы для работы с данными, а Database отвечает за инициализацию базы и предоставление доступа к DAO.

Чтение данных из базы организовано таким образом, чтобы пользовательский интерфейс автоматически реагировал на изменения. Для этого DAO возвращает Flow или LiveData, и при любом обновлении таблицы список сообщений на экране ленты обновляется без дополнительных действий со стороны UI.

Центральным элементом слоя данных является репозиторий. Репозиторий инкапсулирует логику работы с сетью и базой данных и выступает единственной точкой доступа к данным для ViewModel. ViewModel не знает, откуда именно поступают данные, — из сети или из

локального хранилища, — что повышает гибкость и расширяемость архитектуры.

```
class MessageRepository(  
    private val context: Context,  
    private val dao: MessageDao,  
    private val api: MessageApi  
) {  
  
    1 Usage  
    fun getMessages(): Flow<List<MessageEntity>> {  
        return dao.getAll()  
    }  
  
    2 Usages  
    suspend fun refresh() {  
        val remoteMessages = api.getMessages()  
  
        remoteMessages.forEach { dto ->  
            val existing = dao.getById( id = dto.id)  
            if (existing == null) {  
                dao.insert(  
                    MessageEntity(  
                        id = dto.id,  
                        title = dto.title,  
                        body = dto.body,  
                        liked = false  
                    )  
                )  
            } else {  
                dao.updateContent( id = dto.id, title = dto.title, body = dto.body)  
            }  
        }  
    }  
}
```

Рисунок 6 - MessageRepository

5 Экран ленты сообщений и пользовательские сценарии

Экран ленты сообщений является центральным экраном приложения. Для отображения списка используется `RecyclerView`, который оптимально подходит для работы с динамическими наборами данных. `RecyclerView` переиспользует элементы интерфейса, что позволяет эффективно работать даже с большим количеством сообщений.

Для `RecyclerView` был реализован адаптер и `ViewHolder`, отвечающие за связывание данных сообщения с элементами разметки. Каждый элемент списка отображает текст сообщения и дополнительные элементы интерфейса.

На экране предусмотрена возможность ручного обновления данных. При нажатии на кнопку обновления `ViewModel` инициирует вызов метода репозитория, который выполняет загрузку данных из сети и обновляет локальное хранилище. После этого UI автоматически получает обновлённый список.



Рисунок 7 - Интерфейс новостной ленты

Особое внимание было уделено обработке ошибок. В случае отсутствия интернет-соединения или ошибки запроса исключение перехватывается во ViewModel и передаётся в UI через LiveData. Пользователю отображается диалоговое окно с уведомлением об ошибке при отсутствии интернета, если нажали на кнопку обновления.

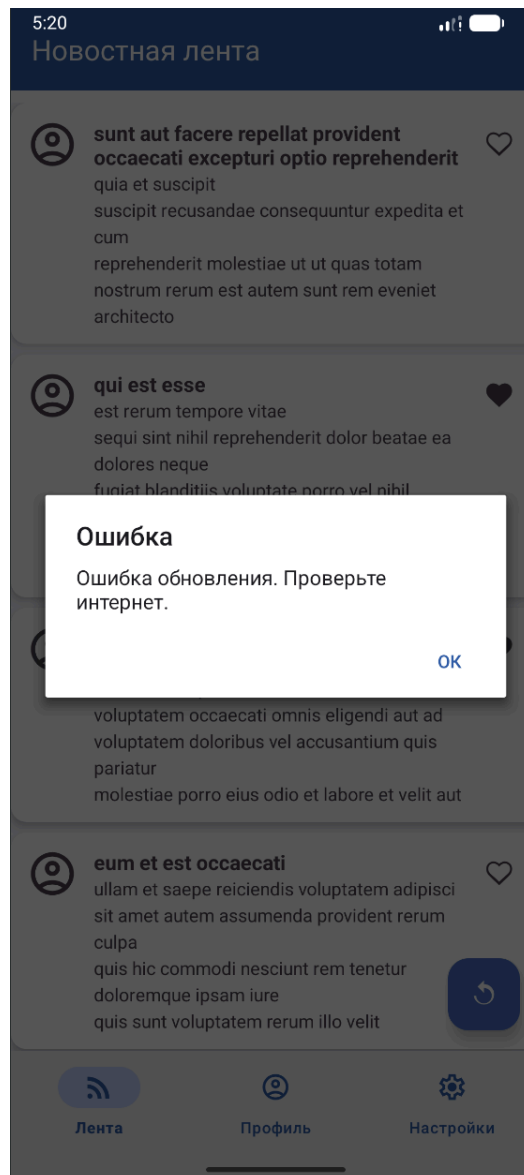


Рисунок 8 - Алерт при обновлении без интернета

6 Улучшение интерфейса и фоновая синхронизация

На завершающем этапе разработки приложение было дополнено улучшенным пользовательским интерфейсом и фоновыми механизмами обновления данных. Для соответствия рекомендациям Material Design были использованы компоненты AppBarLayout и FloatingActionButton. FloatingActionButton применяется для выполнения основного действия на экране — обновления списка сообщений.

Дополнительно была реализована возможность взаимодействия с элементами списка (например, отметка сообщений). Состояние таких действий сохраняется в локальной базе данных, что позволяет корректно отображать их даже после перезапуска приложения или в офлайн-режиме.

Для автоматического обновления данных был настроен WorkManager. С его помощью реализован периодический воркер, который выполняет синхронизацию сообщений при наличии интернет-соединения. Воркер учитывает системные ограничения и корректно обрабатывает ошибки, возвращая результат выполнения в соответствии с состоянием задачи.

При успешной синхронизации приложение отображает системное уведомление. Для этого был создан отдельный канал уведомлений, а при первом запуске приложения реализован запрос разрешения на отправку уведомлений в соответствии с требованиями современных версий Android.

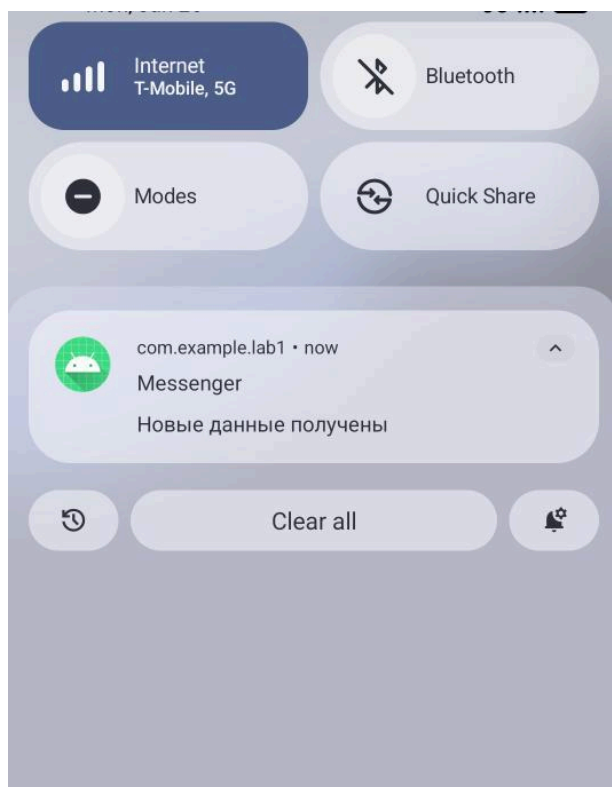


Рисунок 9 - Уведомление при загрузке данных

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы было разработано Android-приложение «Мессенджер», реализующее базовый, но архитектурно корректный функционал обмена сообщениями. В ходе работы были изучены и применены современные инструменты и подходы Android-разработки, включая Navigation Component, архитектуру MVVM, ViewModel, LiveData, Room, Retrofit и WorkManager.

Приложение демонстрирует устойчивость к изменениям конфигурации, корректную работу при отсутствии сети и использование фоновых механизмов обновления данных. Реализованная архитектура обеспечивает удобство сопровождения и расширения проекта, что делает его пригодным для дальнейшего развития.